

Chapitre 3 Types

En C++, il existe deux familles de types prédéfinis, entiers et réels. Les entiers peuvent être strictement positifs (entiers non signés, identifiés par le mot C++ réservé `unsigned`), ou prendre des valeurs positives et négatives (entiers signés, pouvant être identifiés par le mot réservé `signed`).

Le tableau suivant présente la taille et la portée des types prédéfinis de chaque famille.

	Type	Occupation mémoire	Portée
Entier	char	1 octet	−128 à +127
	unsigned char		0 à 255
	short	2 octets	−32 768 à +32 767
	unsigned short		0 à 65 535
	int	n octets	−2 147 483 648 à +2 147 483 647
	unsigned int		0 à 4 294 967 295
	long	4 octets	−2 147 483 648 à +2 147 483 647
	unsigned long		0 à 4 294 967 295
Réel	float	4 octets	$-3.4 \cdot 10^{-38}$ à $+3.4 \cdot 10^{+38}$
	double	8 octets	$-1.797\,693\,134\,862\,3158 \cdot 10^{-308}$ à $+1.797\,693\,134\,862\,3158 \cdot 10^{+308}$
	long double	10 octets	$-3.4 \cdot 10^{-4932}$ à $3,4 \cdot 10^{+4932}$

Tableau 1 : types prédéfinis du C++.

Types entiers

Un nombre entier est un nombre sans virgule qui peut être exprimé dans 3 bases :

- Base **décimale** : l'entier est représenté par une suite de chiffres unitaires (de 0 à 9) ne devant pas commencer par le chiffre 0 (réservé pour indiquer la base octale)
- Base **hexadécimale** : l'entier est représenté par une suite d'unités (de 0 à 9 et de A à F). On indique cette base en faisant précéder le nombre par 0x. Exemple : `int nN = 0x3F4C;`
- Base **octale** : l'entier est représenté par une suite d'unités (de 0 à 7). Les entiers exprimés en base octale commencent par 0 pour les distinguer des entiers en base décimale.

Les entiers sont signés par défaut (ils comportent un signe pour différencier les nombres positifs des nombres négatifs). Le mot réservé `unsigned` est utilisé pour spécifier des entiers non signés, c'est-à-dire strictement positifs. La portée est modifiée en conséquence.

Le suffixe `u` ou `U` indique que la valeur est signée, et le suffixe `l` ou `L` indique que la valeur est de type long. Exemple 1 : `unsigned short s = 7u;` Exemple 2 : `long l = 7L ;`

Type entier `char`

Le type entier `char` (pour character en anglais) permet de stocker 256 valeurs différentes, qui correspondent aux valeurs ASCII de caractères simples.

Codes ASCII

Un code ASCII est entier interprété de la même manière quelle que soit le système utilisé (ordinateur / téléphone, Windows / Unix / Mac Os).

Les 128 caractères des premiers codes sont fixes (voir Tableau 2 page 17).

- Les 32 premiers codes, de 0 à 31, sont des caractères de contrôle qui ne sont pas imprimables. Par exemple le code 13 représente un retour à la ligne, et le code 7 fait produire un bip à certains ordinateurs.
- À partir du code 32, les caractères correspondent successivement aux signes de ponctuation courants, aux opérateurs mathématiques comme, aux chiffres de 0 à 9, puis aux 26 lettres de l'alphabet latin, en capitales puis en minuscules. Les codes 65 à 90 représentent les lettres de A à Z en majuscule, et les codes 97 à 122 les mêmes lettres en minuscule, de a à z.
- Pour passer d'une lettre majuscule à sa minuscule, il suffit d'ajouter 32 en décimal au code ASCII de la lettre majuscule. Exemple : 65 (caractère A) + 32 = 97 (caractère a).

Les 128 codes suivants, appelés codes ASCII étendus, sont personnalisables. Ils servent à définir des codages plus complets, qui ajoutent à l'ASCII les caractères qui manquent à une langue, un pays ou une culture (notamment les caractères accentués qui ne sont pas disponibles dans les 128 premiers caractères, choisis par des Américains dont la langue ne comporte pas d'accents).

Format interne et valeur

Les valeurs des variables sont enregistrées en mémoire sous une forme binaire (suite de N bits b_n , $n \in \mathbb{N}$, qui prennent soit la valeur 0, soit la valeur 1), qui définit son format interne. Ce format interne fixe la portée de la variable, car un entier non négatif à N bits ne peut être supérieur à $2^N - 1$. Par exemple, avec N=8, la portée des entiers non négatifs est $[0..255=2^8-1]$.

Pour les entiers non signés codés sur un octet, on passe de la valeur binaire à la valeur décimale au moyen de la relation :

$$v = \sum_{n=0}^{N-1} b_n \cdot 2^n$$

Chaque bit correspond à la valeur indiquée dans le tableau suivant :

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0

Exemple : conversion en nombre décimal du nombre binaire (01000001)

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
0	1	0	0	0	0	0	1

$$v = 0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

$$v = 0 \cdot 128 + 1 \cdot 64 + 0 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 0 \cdot 2 + 1 \cdot 1$$

$$v = 64 + 1 = 65$$

N°	Char
0	Null
1	Start of heading
2	Start of text
3	End of text
4	End of transm.
5	Enquiry
6	Acknowledge
7	Bell
8	Back space
9	Tabulation
10	Line feed
11	Vertical tabulat.
12	Form feed
13	Carriage return
14	Shift out
15	Shift in
16	Data link escape
17	Device control 1
18	Device control 2
19	Device control 3
20	Device control 4
21	Negative ackn.
22	SYN
23	End of transmiss.
24	Cancel
25	End of medium
26	Substitute
27	Escape
28	File separator
29	Group separator
30	Record separator
31	Unit separator
32	Space
33	!
34	"
35	#
36	\$
37	%
38	&
39	'
40	(
41	)
42	*
43	+
44	,
45	-
46	.
47	/
48	0
49	1
50	2
51	3
52	4
53	5
54	6
55	7
56	8
57	9
58	:
59	;
60	<
61	=
62	>
63	?

Code	Char
64	@
65	A
66	B
67	C
68	D
69	E
70	F
71	G
72	H
73	I
74	J
75	K
76	L
77	M
78	N
79	O
80	P
81	Q
82	R
83	S
84	T
85	U
86	V
87	W
88	X
89	Y
90	Z
91	[
92	
93	]
94	^
95	_
96	`
97	a
98	b
99	c
100	d
101	e
102	f
103	g
104	h
105	i
106	j
107	k
108	l
109	m
110	n
111	o
112	p
113	q
114	r
115	s
116	t
117	u
118	v
119	w
120	x
121	y
122	z
123	{
124	
125	}
126	~
127	Suppression

Code	Char
128	€
129	•
130	,
131	f
132	„
133	...
134	†
135	‡
136	^
137	‰
138	Š
139	ˋ
140	Œ
141	•
142	Ž
143	•
144	•
145	‘
146	’
147	"
148	"
149	•
150	—
151	—
152	~
153	™
154	š
155	›
156	œ
157	•
158	ž
159	Ÿ
160	
161	i
162	ç
163	£
164	¤
165	¥
166	¦
167	§
168	"
169	©
170	ª
171	"
172	¬
173	-
174	®
175	-
176	°
177	±
178	²
179	³
180	´
181	µ
182	¶
183	·
184	,
185	¹
186	º
187	"
188	¼
189	½
190	¾
191	¿

Code	Char
192	À
193	Á
194	Â
195	Ã
196	Ä
197	Å
198	Æ
199	Ç
200	È
201	É
202	Ê
203	Ë
204	Ì
205	Í
206	Î
207	Ï
208	Ð
209	Ñ
210	Ò
211	Ó
212	Ô
213	Õ
214	Ö
215	×
216	Ø
217	Ù
218	Ú
219	Û
220	Ü
221	Ý
222	Þ
223	ß
224	à
225	á
226	â
227	ã
228	ä
229	å
230	æ
231	ç
232	è
233	é
234	ê
235	ë
236	ì
237	í
238	î
239	ï
240	ð
241	ñ
242	ò
243	ó
244	ô
245	õ
246	ö
247	÷
248	ø
249	ù
250	ú
251	û
252	ü
253	ý
254	þ
255	ÿ

Tableau 2 : table des codes ASCII.

Codage binaire

Le code binaire utilise la base 2. La plus petite quantité d'information du code s'appelle bit (pour « binary digit »). Un bit est noté par un b minuscule. Un bit est simple à représenter physiquement : par exemple présence ou absence de signal électrique, de signal lumineux, de champ magnétique, etc.

Dans un nombre binaire, la valeur d'un bit, appelée poids, dépend de la position du bit en partant de la droite. A la manière des dizaines, des centaines et des milliers pour un nombre décimal, le poids d'un bit croît d'une puissance de deux en allant de la droite vers la gauche.

Dans un nombre binaire, la valeur d'un bit, appelée poids, dépend de la position du bit en partant de la droite. A la manière des dizaines, des centaines et des milliers pour un nombre décimal qui croissent d'une puissance de 10 en allant de la droite vers la gauche, le poids d'un bit croît d'une puissance de deux en allant de la droite vers la gauche.

Pour convertir un mot binaire en nombre décimal, il suffit de multiplier la valeur de chaque bit par son poids, puis d'additionner chaque résultat.

L'octet (pour « byte ») est un regroupement composé de 8 bits. Il est noté par un B majuscule. Ce regroupement de nombres par série de 8 est très utilisé car plus lisible. Il a le même objet que la séparation des nombres en base décimale en groupe de 3 nombres pour mieux distinguer les milliers, les millions, etc. 1 256 245 est par exemple plus lisible que 1256245.

Un groupe de 16 bits est appelée mot (en anglais word), noté W.

Codage hexadécimal

Le codage hexadécimal fournit une représentation simplifiée des nombres binaires. Chaque nombre binaire est regroupé en ensembles de 4 bits consécutifs, codés par un chiffre hexadécimal. 4 bits permettant de coder 16 valeurs, on ajoute 6 valeurs aux chiffres décimaux : les lettres majuscules A à F.

Dans la pratique, la conversion binaire vers hexadécimal ou hexadécimal est très simple, au moyen d'une table de conversion hexadécimal ↔ binaire.

Binaire	Hexa	Binaire	Hexa	Binaire	Hexa	Binaire	Hexa
0000	0	0100	4	1000	8	1100	C
0001	1	0101	5	1001	9	1101	D
0010	2	0110	6	1010	A	1110	E
0011	3	0111	7	1011	B	1111	F

Tableau 3 : table de correspondance code binaire - code hexadécimal.

Exemple : le nombre binaire 0100 1101 s'écrit 4D en hexadécimal.

Initialisation d'une variable de type `char`

En C et C++, les variables entières de type `char` peuvent être initialisées de trois manières, en utilisant au choix :

- une valeur décimale qui correspond au code ASCII souhaité.
Exemple : `unsigned char c1 = 80; // initialisation par l'entier 80 équivalent au caractère 'P'`
- un caractère placé entre deux apostrophes.
Exemple : `unsigned char c2 = 'o'; // initialisation par le caractère 'o' qui correspond au code ASCII 111`



Il n'existe pas en C++ de type de données pour les chaînes de caractères (suite de caractères). Pour créer une chaîne de caractères, on utilise des tableaux (ensemble de plusieurs éléments identiques) contenant un caractère dans chacun de ses éléments.

Affichage d'une valeur de type `char`

L'opérateur d'insertion dans le flux `<<` convertit les entiers d'un octet pour afficher le caractère ASCII correspondant.

Exemple

```
#include <iostream>
using namespace std;

int main (void)
{
    char c1 = 80;
    char c2 = 'o';

    cout << c1 << c2 << endl; // affiche les caractères P et o à l'écran
}
```



Attention de ne pas confondre l'utilisation d'un caractère, indiqué entre deux apostrophes `'P'` avec celle d'une chaîne de caractère (ensemble de caractères), placée entre guillemets `"Polytech"`.

Opérations sur les caractères

De nombreuses fonctions sont utilisées avec les caractères. Les plus courantes sont :

- `char toupper('c')` : pour convertir le caractère minuscule `c` en majuscule
- `char tolower('C')` : pour convertir le caractère majuscule `C` en minuscule

Types entier `short`, `int`, `long`

Les type entier `short`, `int`, `long`, qu'ils soient `signed` ou `unsigned`, s'utilisent simplement en fonction de la plage des valeurs stockées dans les variables correspondantes.

Opérations sur les entiers

Outre les opérations arithmétiques usuelles (voir paragraphe « Opérations sur les variables » page 14), on dispose de l'opérateur modulo `%` qui retourne le reste de la division entière de deux entiers.

Exemple : $9 \% 7 = 2$.

Par ailleurs, il faut faire attention à la division `/` de deux entiers qui effectue une division entière. Exemple : $9 / 7 = 1$ (et non 1,286)

9	7
2	1
(9%7 = 2)	(9/7 = 1)

Taille du type entier `int`

En C++, la taille du type entier `int` varie selon le processeur de l'ordinateur utilisé. Cette taille est égale au nombre de bits du bus de données du processeur.

Processeur	Bits du bus de données	Taille de <code>int</code>
8 bits	8	1 octet
16 bits	16	2 octets
32 bits	32	4 octets
64 bits	64	8 octets

Tableau 4 : taille de l'entier `int` en fonction de la taille du bus de données du microprocesseur.

La majorité des processeurs utilisés aujourd'hui dans les ordinateurs ou les téléphones sont des processeurs 64 bits. Le nombre d'octets du type `int` est donc couramment égal à 8 (64 bits / 8 bits = 8 octets, car 1 octet représente 8 bits).

La taille du type `int` est aujourd'hui très souvent supérieure à celle du type `long`.

La variation de la taille du type `int` en fonction du matériel peut apporter des difficultés de programmation. C'est la raison pour laquelle les langages de programmation postérieurs au C++ (java, C#, ...) disposent d'entiers de tailles fixes.

Fonction `sizeof` ()

La fonction `sizeof` () utilisée sur un type `sizeof (type)` ou une variable `sizeof (variable)` retourne la taille en octets du type ou de la variable.

Exemple

```
double dD = 2.718;
cout << sizeof(dD) << " octets."; // affiche 8 octets (double)
cout << sizeof(short) << " octets."; // affiche 2 octets (short)
```



Utilisez les instructions `cout << sizeof(int);` dans votre programme pour déterminer le nombre de bits de données du processeur de votre ordinateur.

Types réel `float`, `double`, `long double`

Les valeurs réelles sont constituées d'une partie entière, d'une partie décimale, éventuellement complétée d'un exposant, signé ou non, précédé de la lettre e ou E.

La partie entière ou la partie décimale peut être omise, mais pas les deux.

Le suffixe f ou F indique une valeur de type `float`.

Le suffixe l ou L indique une valeur de type `long double`. Sans suffixe, la valeur est de type `double`.

Exemple

```
double dD = .5E1 ; // double égal à 5.0)
float fF = 50e-1F ; // float égal à 5.0
long double lD = 5.0L ; // long double égal à 5.0
```

Format interne des réels

Les nombres réels sont des nombres à virgule flottante, c'est-à-dire des nombres dans lequel la position de la virgule n'est pas fixe, mais est repérée par une partie de ses bits (appelée l'exposant), le reste des bits permettant de coder le nombre sans virgule (la mantisse).

Les nombres de type `float` sont codés sur 32 bits dont :

- 23 bits pour la mantisse
- 8 bits pour l'exposant
- 1 bit pour le signe

±	exposant								mantisse																							
b ₃₁	b ₃₀	b ₂₉	b ₂₈	b ₂₇	b ₂₆	b ₂₅	b ₂₄	b ₂₃	b ₂₂	b ₂₁	b ₂₀	b ₁₉	b ₁₈	b ₁₇	b ₁₆	b ₁₅	b ₁₄	b ₁₃	b ₁₂	b ₁₁	b ₁₀	b ₉	b ₈	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀	
31	30							23	22																							0

De la même manière, les nombres de type `double` sont codés sur 64 bits dont :

- 52 bits pour la mantisse
- 11 bits pour l'exposant
- 1 bit pour le signe

Les nombres de type `long double` sont codés sur 80 bits dont :

- 64 bits pour la mantisse
- 15 bits pour l'exposant
- 1 bit pour le signe



Utilisez Wikipédia (par exemple) pour expliquer le mécanisme de conversion des données réelles codées en binaire.

La précision des nombres réels est approchée. Elle dépend du nombre de positions décimales, suivant le type de réel elle sera au moins :

- de 6 chiffres après la virgule pour le type `float`
- de 15 chiffres après la virgule pour le type `double`
- de 17 chiffres après la virgule pour le type `long double`

Type booléen

Comme le C, le C++ ne définit pas le type booléen comme type natif du langage. Le type `int` est utilisé pour gérer les variables binaires, qui peuvent prendre les deux valeurs `true` et `false`, qui correspondent à une conversion implicite valant respectivement 1 ou 0. (en réalité toute autre valeur que 0 est considérée comme égale à `true`).

Certains compilateurs autorisent néanmoins l'usage des types `bool` ou `boolean`, qui dans la plupart du temps correspondent à une redéfinition du type `int`.

Opérations sur les booléens

Bit b ₁	Bit b ₂	b ₁ && b ₂ (et)	b ₁ b ₂ (ou)	b ₁ ^ b ₂ (ou excl.)	! b ₁ (non)	b ₁ == b ₂ (=)	b ₁ != b ₂ (≠)
0 (F)	0 (F)	0 (F)	0 (F)	0 (F)	1 (V)	1 (V)	0 (F)
0 (F)	1 (V)	0 (F)	1 (V)	1 (V)	1 (V)	0 (F)	1 (V)
1 (V)	0 (F)	0 (F)	1 (V)	1 (V)	0 (F)	0 (F)	1 (V)
1 (V)	1 (V)	1 (V)	1 (V)	0 (F)	0 (F)	1 (V)	0 (F)

Tableau 5 : opérations sur les booléens (V ≡ VRAI, F ≡ FAUX).

Habitudes de nommage des variables

Choisir des identificateurs de variables pertinents est assez difficile. Par identificateurs pertinents, on entend des identificateurs suffisamment explicites pour représenter la variable sans confusion, mais également suffisamment courts pour ne pas nuire à la lecture ni à la compréhension des programmes :

- L'identificateur doit être immédiatement compréhensible
- Éviter les abréviations qui ne sont que très rarement compréhensibles

Les programmeurs C++ ont l'habitude de mettre en majuscule le premier caractère de leurs variables. Il est aussi d'usage d'utiliser un caractère minuscule pour « référencer » le type de la variable dans l'identificateur de la variable : `c` pour `char`, `s` pour `short`, `n` pour `int`, `f` pour `float`, `d` pour `double`, et, nous le verrons plus tard, `p` pour pointeur, `p` pour tableau, `o` pour objet, etc. `u` et `l` sont également ajoutés pour les variables `unsigned` ou `long`.

On aboutit ainsi à la règle de nommage suivante :

```
type tVariable = valeur;
```

Exemples :

```
char cC = 'P';           // variable caractère
unsigned short usS = 34U; // variable unsigned short
int nN = 3;              // variable entière
long int lnL = 670L;     // variable entier long
double dD = 2.718;       // variable double
```

Exception :

Les indices de boucle sont notés par des caractères uniques à partir de la lettre `i` : `i`, `j`, `k`, `l`, `m`, `n`, ...

Types utilisés

La capacité mémoire des ordinateurs et des smartphones disponibles aujourd'hui tend à voir généraliser l'usage des seuls types :

- `char` pour gérer les caractères
- `int` pour manipuler les entiers (signés et non signés)
- `double` pour tous les réels



Limitez l'usage de ces seuls 3 types dans vos programmes.

Création de type

En C++ (comme en C), il est possible de définir de nouveaux types de données grâce au mot réservé `typedef`.

Syntaxe :

```
typedef type NouveauType;           // définition de type
```

où

- `type` est un type de données prédéfini (`char`, `short`, `int`, `long`, `signed` ou `unsigned`, `float`, `double`, `long double`, ...) ou un type défini par le programmeur
- `NouveauType` définit le nom attribué au nouveau type de donnée

Exemple

```

#include <iostream>
using namespace std;

int main (void)
{
    typedef prix double;

    prix prPomme = 1.25;           // prix du kilo de pommes
    prix prPoire = 1.40;           // prix du kilo de poires
    double dPoids = 2,7;           // poids de fruits
    cout << "Prix des poires : " << prPoire * dPoire << endl;
}

```

Redéfinir des types prédéfinis permet de modifier simplement un type redéfini. Dans l'exemple ci-dessus, changer `typedef prix double;` en `typedef prix float;` passe très facilement le stockage des prix sur 8 octets (`double`) à 4 octets (`float`), sans interaction avec la variable `dPoids` qui reste de type `double`.

Conversion de type (`cast`)

La conversion de type transforme le type d'une valeur en un autre type, différent du type initial. Les conversions de types sont assez courantes.

On distingue deux types de conversions. Les conversions implicites, effectuées automatiquement par le compilateur ; les conversions explicites, forcées par le programmeur, appelées `cast` :

- conversion **implicite** : le compilateur convertit implicitement le type d'une donnée pour éviter une indétermination ou une erreur.
Exemple : `double dD = 3; // 3 (int) devient 3.0 (double) affecté à dD`
- conversion **explicite** : la conversion est réalisée par une opération de `cast`. Il suffit d'indiquer entre parenthèses le nouveau type souhaité pour la valeur indiquée.
Exemple : `cout << (int) 2.718; // affiche 2`

Quand les conversions implicites ne sont pas possibles sans ambiguïté, le compilateur indique une erreur.

Exemple : `int nN = 3.567; // erreur car possible perte de données (partie décimale)`

Écrire : `int nN = (int) 3.567; // pas d'erreur car la conversion est explicite`

Le langage C++ autorise une notation fonctionnelle pour les conversions explicites.

Exemple : `int nN = int(3.567); // cast par notation fonctionnelle`